

ESWIN

ENNP用户手册

Rev 3.0

2026-4-13

声明

知识产权声明

除非另有说明，ESWIN 保留对本产品的所有知识产权。未经 ESWIN 授权，不得对本产品进行反向工程、反编译，不得修改、改编、制作、分发本产品的衍生作品。

ESWIN 保留更新本文档或改进其中所提及的产品或服务的权利。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证或承诺，无论明示或默示。

“ESWIN”商标和其他 ESWIN 标识，为北京奕斯伟计算技术股份有限公司及（或）其母公司所有的商标，未经授权，不得擅自使用。

责任限制

除本文档或交易文书另有约定外，ESWIN 不提供任何明示、暗示或法律上的保证，包括但不限于适销性、适用性、所有权或非侵权的任何暗示或明示保证，或因任何建议、规格、样品、交易、惯例或贸易惯例而产生的任何保证。

ESWIN 不对任何意外性、惩罚性、间接性的损害承担责任，包括但不限于利润损失、业务中断或采购任何替代商品或服务而产生的成本。

修订记录

文档版本	日期	说明
v3.0	2026/04/13	重构文档
v2.1	2025/10/21	修订内容： 1. 修订了关于 ESWIN Model Zoo 相关的内容 影响范围： 本版本更新，删除了第六章中关于 ESWIN Model Zoo 内容与代码仓库中 README 文件重复的部分，包含其环境配置、功能示例等内容。
v2.0	2025/09/30	修改 EsQuant 量化工具链介绍，增加了对自定义预处理、混合精度量化 (fp16) 的支持。
v1.3	2025/01/21	新增 EsAAC 编译选项 -lutdsp 的说明。
v1.2	2025/01/06	修订内容： 1. 新增 EsAAC 对 ViT 模型的编译选项 -enable-uosp 说明。 影响范围： 本版本新增了编译选项说明，对其余章节无增删。用户在编译 ViT 模型时，请添加编译选项 -enable-uosp；其余场景下，用户延续之前使用方式即可。
v1.1	2024/12/26	修订内容： 1. 新增 EsAAC 混合模型生成方式说明； 2. 增加用户自定义 dsp 算子开发示例； 3. 完善了 dsp 支持算子列表。
v1.0	2024/11/18	增加用户量化流程使用自定义预处理函数的说明。
v0.9.2	2024/10/22	修订了 NPU Runtime 实现动态 Batch size 和复合模型的说明。
v0.9	2024/10/12	增加 NPU Runtime 实现动态 Batch size 和复合模型说明。
v0.8	2024/09/10	完善 yolov3 示例说明。
v0.7	2024/08/28	增加从 GitHub 导出 ONNX 模型的示例。
v0.6	2024/08/09	完善 docker 的使用说明。
v0.5	2024/07/17	修改章节顺序，修改 EsAAC 和 EsQuant 工具说明。
v0.4	2024/03/25	修改 NPU Runtime 概要描述和调用图等。
v0.3	2024/02/22	初始版本

目录

声明	1
知识产权声明	1
责任限制	1
修订记录	2
1. 概述	4
1.1. ENNP 简介	4
1.2. 软件开发流程	4
1.2.1. 离线工具链使用 (PC 端)	4
1.2.2. 在线开发验证 (RISC-V 端)	4
2. EsQuant 使用指南	6
2.1. EsQuant 工具介绍	6
2.2. 模型量化说明	6
2.2.1. 量化配置文件说明	6
2.2.2. 量化命令说明	8
2.3. 量化精度分析说明	9
3. EsAAC 使用指南	10
3.1. EsAAC 工具介绍	10
3.2. 生成模型功能介绍	10
3.2.1. 命令行参数详解	10
4. NPU Runtime 开发指南	12
4.1. Runtime 框架	12
4.2. 输入模型要求	12
4.3. C 语言 API 开发指南	12
4.3.1. 核心开发流程 (C)	13
4.3.2. 代码示例 (C)	13
4.4. Python API 开发指南	14
4.4.1. 核心开发流程 (Python)	14
4.4.2. 代码示例 (Python)	14
4.5. 调试与性能评测	15
4.5.1. 日志调试	15
4.5.2. es_run_model 工具	15
5. AcceleratorKit 开发指南	16
5.1. 软件框架	16
5.2. 调用流程	17
5.2.1. 依赖模块	17
5.2.2. 开发步骤	17
6. DSP 算子开发指南	18
6.1. Sample 文件说明	18
6.2. 搭建 ESWIN 交叉编译环境	18
6.2.1. 获取 riscv 交叉编译工具	19
6.2.2. 准备 eswin 交叉编译环境	19
6.3. Cadence Vision Q7 DSP 编译环境搭建	19
6.4. 自定义算子开发	19
6.4.1. host 侧准备	19
6.4.2. DSP 侧算子开发	20
6.5. 算子编译	21
6.6. 自定义算子测试	21
6.7. DSP 支持算子列表	21

1. 概述

1.1. ENNP 简介

ENNP (ESWIN Neural Network Processing) 是奕斯伟媒体处理芯片的智能计算异构加速平台。开发者可基于 ENNP 提供的 API 接口，调用 EIC7700/EIC7702 芯片内部的 **NPU**、**DSP**、**HAE** 等异构计算单元，实现神经网络推理、图像变换处理及其他自定义硬件加速功能。

ENNP 平台包含**离线开发工具套件**和**运行时软件框架**，支持从模型量化、编译到部署的全流程开发。其整体软件栈架构如下图所示：

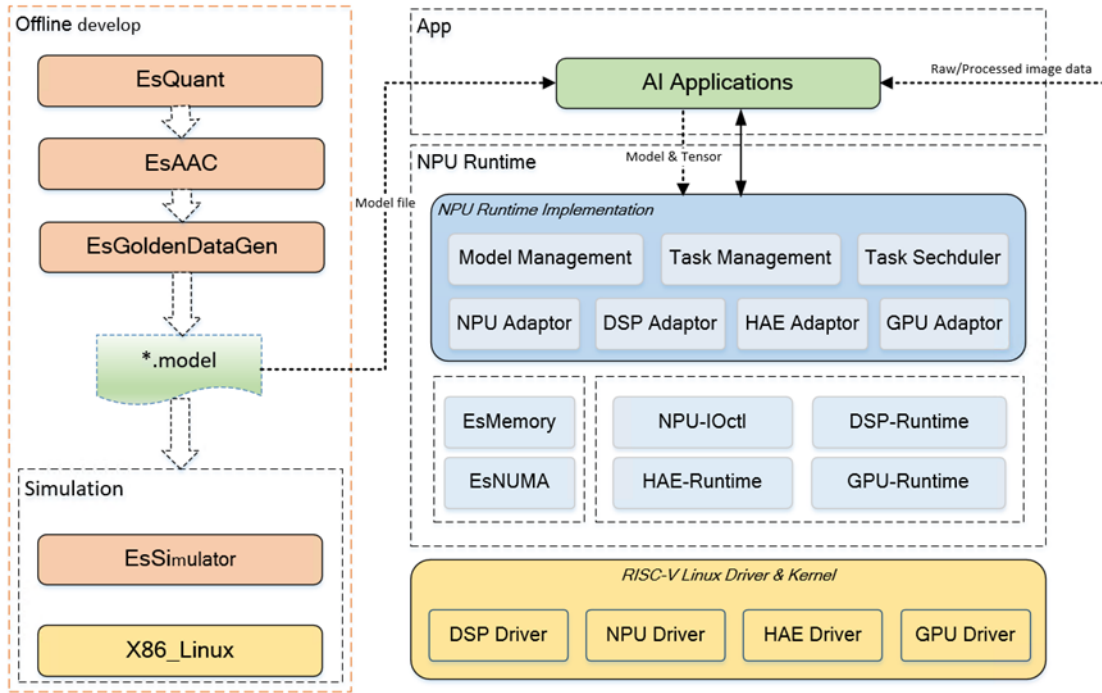


图 1: ENNP 软件栈框图

1.2. 软件开发流程

基于 ENNP 的开发流程可细分为 **离线工具链使用** 与 **在线开发验证** 两个阶段。

1.2.1. 离线工具链使用（PC 端）

此阶段主要在 x86 平台上进行，利用以下工具完成模型的预处理与验证：

1. **环境配置**：安装 ENNP 所需的工具套件（Docker 镜像）。
2. **模型获取**：利用 PyTorch/TensorFlow 训练网络，并导出为 ONNX 格式（.onnx 文件）。
3. **模型量化**：使用 **EsQuant** 基于 ONNX 文件与量化配置文件，生成量化参数表（table.json）。
4. **模型编译**：使用 **EsAAC** 工具基于 ONNX 文件和量化表，编译生成硬件可执行的离线模型（.model 文件）。
5. **仿真验证**：使用 **EsGoldenDataGen** 生成参考数据，并通过 **EsSimulator** 进行 PC 端仿真，确保模型精度符合预期。

1.2.2. 在线开发验证（RISC-V 端）

此阶段在 EIC7700/EIC7702 开发板上进行，验证模型在真实硬件上的表现：

1. **代码开发**：基于 ESSDK 提供的 C 或 Python API，编写应用程序。
2. **交叉编译**：使用 RISC-V 交叉编译器编译上述代码。

3. **执行验证：**将程序拷贝至开发板运行，实测推理效果和性能指标（FPS/延时）。

通过上述流程，开发者可以高效完成从算法模型到芯片部署的全链路开发。

2. EsQuant 使用指南

2.1. EsQuant 工具介绍

EsQuant 是基于 Python 接口的开源模型量化工具。其核心优势在于允许用户自定义预处理函数（Preprocessing Function），同时预置了 ImagenetOpencv、ImagenetTorch 和 Yolo 等常用预处理函数，极大地简化了分类模型和检测模型的量化工作流程。

EsQuant 的主要功能包括：

- **模型量化**：支持将浮点模型（Float32）量化成定点模型（Int8/Int16），支持对称量化，并允许不同网络层采用混合精度量化策略。
- **量化精度分析**：该功能将给出模型量化后每一层推理结果与浮点数推理结果的余弦相似度（Cosine Similarity），包含累计误差分析（Accumulated Error）和重置误差分析（Reset Error），为定位和修复量化精度损失提供数据支撑。

2.2. 模型量化说明

2.2.1. 量化配置文件说明

量化配置文件为 JSON 格式。以 YOLOv3 模型为例，其结构及参数说明如下表所示：

```
{
  "version": "1.3",
  "model": {
    "model_path": "./416x416/pruned-model/yolov3_416_416-cut.onnx",
    "save_path": "./416x416/quantized-model/",
    "images_list": "/IBU_8T/IBU_SOFTWARE/algo_soft/datasets/Coco/cal_1000.txt",
    "analysis_list": "/IBU_8T/IBU_SOFTWARE/algo_soft/datasets/Coco/cal_1.txt"
  },
  "quant": {
    "quantized_method": "per_channel",
    "quantized_dtype": "int8",
    "requant_mode": "mean",
    "quantized_algorithm": "mse_eic",
    "optimization_option": "auto",
    "bias_option": "absmax",
    "nodes_option1": [],
    "nodes_option2": [],
    "mean": [
      0,
      0,
      0
    ],
    "std": [
      1,
      1,
      1
    ],
    "norm": true,
    "scale_path": "",
    "enable_analyse": true,
    "device": "cuda"
  },
  "preprocess": {
    "input_format": "RGB",
    "channel_first": true,
    "keep_ratio": false,
    "resize_shape": [
      416,

```

```

        416
    ],
    "crop_shape": [
        416,
        416
    ]
}
}

```

配置项	示例	详细说明
model_path (必选)	/path/yolov3.onnx	待量化模型的路径。目前仅支持 ONNX 格式，未配置将导致程序报错中止。
save_path (必选)	/path/save/	量化输出文件（如 preprocess.json、quantized.onnx 等）及精度分析结果的保存路径。未配置将报错中止。
images_list (必选)	/path/image_list.txt	Calibration 数据集图片路径列表。建议包含 1000 张图片，用于统计网络推理过程中的数据分布。
analysis_list (必选)	/path/image_list_2.txt	精度分析用的图片路径列表。考虑到量化时间，建议仅提供 1 张数据。
quantized_method	per_channel	量化方式。可选 per_channel（按通道统计）或 per_layer（按层统计），默认 per_channel，目前只支持 per_channel。量化不同 tensor 的数据时（如 conv 的 weight、bias），单独统计各 channel 的数据分布计算 scale（per_channel）分别量化，或者统计所有 channel 的分布统一量化（per_layer）。
quantized_dtype	int8	全局量化数据类型。可选 int8 或 int16，默认 int8。未被 nodes_i8/i16 标记的层将使用该精度。
requant_mode	mean	Requant 策略。可选 mean（取均值）或 max（取最大值），默认 mean。用于统一相邻层的 scale，避免频繁插入 requant 算子带来的开销。
quantized_algorithm	at_eic	量化算法。可选 at_eic（自适应阈值）或 mse_eic。 Yolo 系列推荐 mse_eic，分类模型推荐 at_eic。
optimization_option	auto	模型优化等级。可选 option1、option2 或 auto。大部分情况下 option2 精度和速度更优，auto 会根据网络结构自动调整。
bias_option	absmax	Bias 统计模式。可选 absmax 或 max，默认 absmax，通常在大多数模型中效果更优。
enable_analyse	true	是否开启精度分析。开启则设为 true，否则为 false，默认开启。
device	cuda	运行设备。使用 CPU 填写“cpu”，使用 GPU 填写“cuda”，默认使用 cuda。
nodes_option1	["conv_1"]	对于全局为 option2 或 auto 的网络，可指定特定 Conv 节点强制使用 option1 量化。
nodes_option2	["conv_1"]	对于全局为 option1 或 auto 的网络，可指定特定 Conv 节点强制使用 option2 量化。
nodes_i8	["conv_1"]	对于全局 int16 量化的网络，可将某些层指定为 int8 以加快计算速度。
nodes_i16	["conv_1"]	对于全局 int8 量化的网络，可将精度损失较大的层（尤其是检测模型的输出层）指定为 int16。
nodes_fp16	["conv_1"]	对于全局 int8 量化的网络，必须将某些特殊层（如目标检测的 decode 过程）指定为 fp16 计算。
output_variables_dtype	{"326": "int16", ...}	字典类型，指定输出 Tensor 的数据类型。 Yolov5 建议配置为 int16 以保证检测精度。

配置项	示例	详细说明
mean (必选)	[0.485, 0.456, 0.406]	预处理中的均值参数 (除以 255 后的值)。注意: 即使使用自定义预处理, 该参数也必须配置, 仅用于 Input Blob 的 scale 计算。
std (必选)	[0.229, 0.224, 0.225]	预处理中的方差参数 (除以 255 后的值)。注意: 即使使用自定义预处理, 该参数也必须配置, 仅用于 Input Blob 的 scale 计算。
input_format	BGR	网络所要求的输入图片颜色通道顺序。可选 RGB 或 BGR。
keep_ratio	true	是否在 Resize 过程中保持长宽比。true 时要求 resize_shape 为 1 维向量, false 时为 2 维向量。
norm	true	预处理是否进行归一化 (即先除以 255)。默认 true, 此时均值和方差必须在 0-1 之间。
channel_first	true	根据输入 ONNX 的布局设置。true 为 NCHW, false 为 NHWC。Python 自定义预处理通常无需设置此项。
resize_shape	[1024,2048]	图片缩放的目标尺寸。keep_ratio 为 true 时是 1 维向量 (如 [640]), 为 false 时是 2 维向量 (如 [640, 640])。
crop_shape	[1024,2048]	图片中心截取的目标尺寸。Python 接口中, resize_shape 必须大于或等于 crop_shape 。

2.2.2. 量化命令说明

量化命令的基本格式如下。

使用预置预处理函数：

```
python Example_with_config.py --config_path /config.json --preprocess_name Yolo
```

- **ImagenetTorch**: 用于分类模型, 读入 BGR 的 numpy 数据, 转为 PIL.Image 后采用 torchvision.transforms 接口进行处理。
- **ImagenetOpencv**: 用于分类模型, 读入 BGR 的 numpy 数据, 使用 cv2 接口进行 resize, CenterCrop 和 Normalize 采用 numpy 实现。
- **Yolo**: 用于 Yolo 系列检测模型, 核心逻辑采用 letterbox 处理方式。

使用自定义预处理函数：对于其他图像任务, 客户可以自定义预处理函数。

```
python Example_with_config.py --config_path /config.json --preprocess_name Custom
↪ --custom_preproc_path=datasets/preprocess_custom.py
```

此时需在 config.json 中配置 preprocess.use_custom 为 true , 并指定 dataset.type 为自定义类名。

config.json 配置示例：

```
{
  "version": "1.3",
  "model": {
    ...
  },
  "quant": {
    ...
  },
  "preprocess": {
    "use_custom": "true",
    "dataset": {
      "type": "CustomDataset",

```

```

        "param1": "value1",
        "param2": "value2"
    }
}

```

preprocess_custom.py 中对应类的__init__函数，第一个参数必须为 data_list，其他参数可以根据实际情况进行决定，注意需要和 config.json 中 preprocess.dataset 保持匹配。另外__getitem__返回 numpy 数据时，需要把 batch 维度进行 squeeze 操作。实现如下：

```

import os
import numpy as np
from torch.utils.data import Dataset

class CustomDataset (Dataset):
    def __init__(self, data_list, param1, param2):
        ...

    def __getitem__(self, index):
        ...
        return dict(input=data_x.squeeze(0))

    def __len__(self):
        ...

```

2.3. 量化精度分析说明

该功能通过对比浮点推理和量化推理的逐层数据，计算余弦相似度，并根据结果进行量化精度分析。

执行量化后，会生成两个 TXT 文件：

1. precision_accumulate_result.txt：量化模型和浮点数模型逐层累计的余弦相似度误差。
2. precision_reset_result.txt：量化模型和浮点数模型每层均重置输入获得的最后一层余弦相似度误差。

首先，检查 precision_accumulate_result.txt 文件中最后一个算子节点（即网络最终输出层）对应的 cos_dist(mean) 数值是否大于 95%。若满足阈值，则视为量化误差在可接受范围内；若不满足，则需进一步分析 precision_reset_result.txt 文件。

精度调优流程：

1. **定位问题层：**查看 precision_reset_result.txt，找出余弦相似度显著下降的算子层。
2. **调整量化精度：**在 config.json 配置文件中，将问题层添加至 nodes_i16（或 nodes_fp16）字段中，强制该层使用更高精度（INT16 或 FP16）进行量化。**注意：**切勿修改网络首层和最终输出层的 dtype。
3. **重新量化与分析：**保存配置后重新执行量化与精度分析流程。通常情况下，经过上述调整即可显著提升模型精度。若精度仍未达标，请检查 optimization_option 和 quantized_method 是否遵循建议配置（通常默认配置即为最优）。
4. **自定义预处理排查：**若您使用了自定义预处理函数，请重点排查以下两项：
 - **颜色通道一致性：**确认 input_format 参数设置与自定义预处理函数的实际输出（RGB/BGR）是否一致。
 - **数据布局（Layout）：**EsQuant 默认接收 NCHW 布局的数据，请确认自定义预处理输出与模型输入要求是否匹配。若配置错误，会导致累积误差极低。

当精度指标满足需求后，即可结合量化工具生成的 table.json，quantized.onnx 与模型编译工具，生成可在目标硬件上运行的模型文件。

3. EsAAC 使用指南

3.1. EsAAC 工具介绍

EsAAC (ESWIN AI Accelerator Compiler) 是 ESWIN 面向自研芯片架构打造的深度学习编译器。它能够将主流人工智能框架 (TensorFlow、PyTorch、Caffe、ONNX 等) 训练好的网络模型, 通过解析、优化和映射, 转换成 EIC7700 硬件可加载运行的离线模型 (.model 文件)。EsAAC 集模型转换、硬件优化、代码生成为一体, 旨在最大化利用 NPU、DSP 等异构计算单元的性能。

核心功能特性:

- **广泛模型支持:** 支持图像分类 (如 ResNet、MobileNet)、目标检测 (如 YOLO 系列)、图像分割等主流 CV 模型。
- **多 Batch 编译:** 支持静态多 Batch 模型编译, 最大 Batch Size 可达 16 (具体视模型复杂度而定), 满足高吞吐场景需求。
- **异构计算调度:** 支持配置不同异构计算单元 (NPU、DSP) 的资源占用比例, 实现最优算力分配。
- **SRAM 优化:** 支持配置 SRAM 缓存空间, 利用片上高速存储缓存中间结果, 减少 DDR 访存开销。
- **前后处理定制:** 支持用户定制的前处理 (如图像色彩空间转换、归一化) 和后处理 (如非极大值抑制 NMS) 计算, 无缝集成到推理流水线中。

EsAAC 的编译包含以下三个阶段:

1. **模型解析与初始化:** EsAAC 读取用户输入的预训练模型文件 (如 ONNX 格式), 并结合配置文件与命令行参数启动编译流程。
2. **SRAM 优化调度:** 用户可配置 SRAM 的可用容量。EsAAC 会通过高效算法自动调度与分配 SRAM 空间, 利用片上高速存储缓存中间结果, 从而显著提升访存效率。
3. **硬件映射与代码生成:** EsAAC 对 ONNX 模型进行深度解析, 将计算图精准映射到底层硬件单元 (NPU/DSP), 最终生成可由 Runtime 加载运行的离线模型。该模型以序列化二进制文件形式保存, 默认命名为 <model_name>.model。

3.2. 生成模型功能介绍

EsAAC 提供了一个名为 EsAAC 的可执行文件, 它是模型编译的核心工具。通过命令行参数, 用户可以精确控制编译行为。

3.2.1. 命令行参数详解

下表列出了 EsAAC 的主要命令行参数及其说明:

参数	说明	默认值	是否必选
--input-model	输入模型文件路径 (ONNX 格式)	无	是
--input-nodes	模型的输入节点名称 (若配置此参数, 则必须配置 --input-shapes)	模型自带	否
--input-shapes	模型输入节点的形状, 格式为 [N,C,H,W] 或 [N,H,W,C] (若配置此参数, 则必须配置 --input-nodes)	模型自带形状	否
--output-nodes	模型的输出节点名称	模型自带	否
--output-shapes	模型输出节点的形状 (若配置此参数, 则必须配置 --output-nodes)	无	否
--quant-stats	量化统计文件 (由 EsQuant 生成的 table.json)	无	是 (量化模型)
--equant-config-path	量化配置文件路径 (参考 EsQuant 章节)	无	否
--loadable-name	输出模型文件的名称	输入模型名 + .model	否
--run-weight-csc	是否对 weight 进行 CSC 压缩处理	no	否
--sram-capacity	SRAM 的空间大小, 单位 KB	4096	否

参数	说明	默认值	是否必选
<code>--dsp-core</code>	指定 DSP 的个数 (1~4)	1	否
<code>--inference-datatype</code>	生成模型的数据类型, 可选 DT_QINT8、DT_QINT16	DT_QINT8	否
<code>--gen-fp16-model</code>	是否生成 FP16 推理的模型	yes	否
<code>--reverse-input-channel</code>	是否切换输入模型的通道顺序 (如 BGR<->RGB)	no	否
<code>--input-format</code>	设置输入形状格式是 NCHW 或 NHWC	NCHW	否
<code>--convert-to-qm</code>	设置是否将输入模型转换为 QM 格式, 提供给 EsQuant 使用	no	否
<code>--run-device</code>	指定生成模型的推理硬件类型, 可选 npu、dsp 和 all	npu	否
<code>--enable-uosp</code>	编译 ViT 模型时需加上此选项进行 shape 变换	False	否
<code>--lutdsp</code>	设置是否使用 DSP 的 LUT 算子	False	否

重要说明:

目前通过 `--run-device all` 命令生成 Throughput 模型支持的模型列表如下, 且通过 `--input-model` 传入的文件名需与上述名称保持一致 (如 `resnet50.onnx`):

[Inception_V1, inception-v3, inception-v4, resnet18, resnet50, segnet, squeezenet_v1.0, squeezenet_v1.1, densenet121, densenet201, yolov2, yolov3, senet50, yolov5s]。

4. NPU Runtime 开发指南

NPU Runtime 是 ENNP 平台的核心组件，提供了一套用于在 EIC7700/EIC7702 芯片上加载和执行神经网络模型的 **C/C++ 与 Python API 接口**。它负责模型的生命周期管理、异构硬件（NPU、DSP、HAE）的任务调度、内存管理及异步执行流程。

详细 API 描述请查看 [ENNP 开发者手册](#)。

4.1. Runtime 框架

NPU Runtime 可以直接加载基于 ENNP 工具量化编译的离线模型，用于实现目标识别、图像分类等功能。用户基于 NPU Runtime 开发智能分析方案，由编译器自动分析并优化执行过程，最大化复用 NPU、DSP、HAE 等硬件资源，从而提升硬件利用率并优化系统功耗。

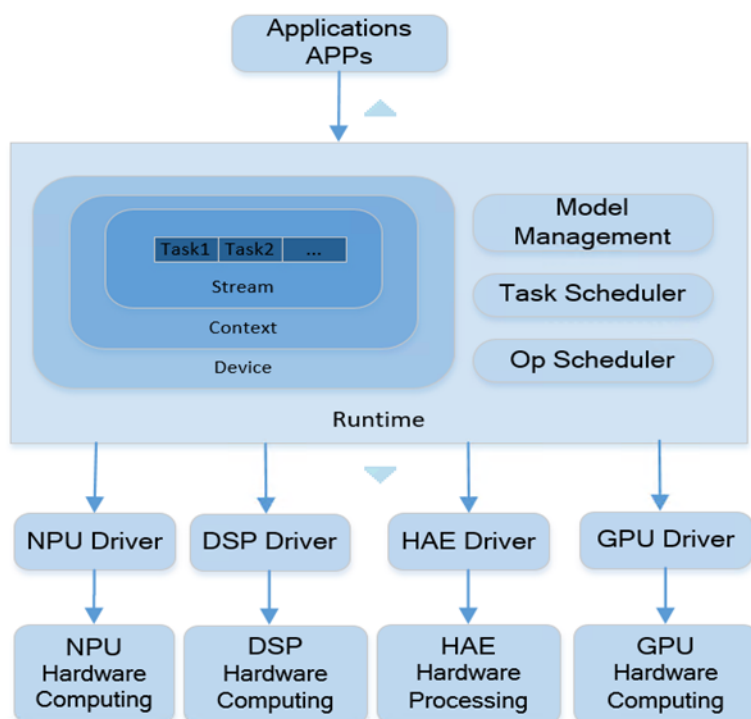


图 2: 运行时框架

核心架构概念：

- **Device**：表示一个 NPU 推理硬件执行单元，不同 Device 上的 Task 是并行执行的。
- **Context**：描述了 NPU 运行的一个配置上下文，管理对应的内存分配及硬件资源；同时作为一个容器，管理在对应 Context 上创建对象的生命周期。一个 Context 可以关联到一个进程的多个线程，不同 Context 的 Stream 是完全隔离的。
- **Stream**：用于维护异步操作任务完成后等待用户取回的队列。
- **Task**：任务的执行载体，包含执行一个任务所需的必要信息。

4.2. 输入模型要求

NPU Runtime **仅支持**由 ESWIN EsAAC 模型编译器产生的离线模型（.model 文件）。在部署到硬件之前，强烈建议使用 EsGoldenDataGen 和 EsSimulator 工具对该模型进行精度和功能性校验。

4.3. C 语言 API 开发指南

C API 提供了最直接、高性能的硬件访问能力，适用于嵌入式应用和底层开发。

4.3.1. 核心开发流程 (C)

C API 的调用遵循“系统初始化 → 设备配置 → 模型加载 → 任务提交 → 资源释放”的逻辑。

1. **系统初始化**: 调用 `ES_SYS_Init()` 初始化系统资源。
2. **设备设置**: 调用 `ES_NPU_SetDevice()` 指定使用的 NPU 设备 ID。
3. **模型加载**: 调用 `ES_NPU_LoadModelFromFile()` 从文件系统加载 `.model` 文件。
4. **创建上下文 (Context)**: 调用 `ES_NPU_CreateContext()`。Context 是资源管理容器, 关联到一个进程的多个线程, 不同 Context 的 Stream 相互隔离。
5. **创建流 (Stream)**: 调用 `ES_NPU_CreateStream()`。Stream 用于维护异步操作任务完成后等待用户取回的队列。
6. **内存分配**: 通过 `ES_SYS_MemAlloc()` 分配输入/输出 Buffer。
7. **任务提交**:
 - **同步**: 调用 `ES_NPU_Submit()` 提交任务并等待完成。
 - **异步**: 调用 `ES_NPU_SubmitAsync()` 提交任务到 Stream。
8. **资源释放**: 推理完成后, 依次调用 `ES_SYS_MemFree()`, `ES_NPU_DestroyStream()`, `ES_NPU_DestroyContext()`, `ES_NPU_UnloadModel()` 和 `ES_SYS_Deinit()` 释放资源。

4.3.2. 代码示例 (C)

```
#include "es_npu_runtime.h"

int main() {
    //1. 系统初始化
    ES_SYS_Init();

    //2. 设置设备
    ES_NPU_SetDevice(0);

    //3. 加载模型
    ES_NPU_ModelHandle model = ES_NPU_LoadModelFromFile("yolov3.model");

    //4. 创建 Context 和 Stream
    ES_NPU_ContextHandle ctx = ES_NPU_CreateContext();
    ES_NPU_StreamHandle stream = ES_NPU_CreateStream(ctx);

    //5. 准备输入输出内存 (伪代码)
    void* input_buffer = ES_SYS_MemAlloc(input_size);
    void* output_buffer = ES_SYS_MemAlloc(output_size);

    //6. 填充输入数据
    fill_input_data(input_buffer);

    //7. 提交推理任务 (同步)
    ES_NPU_Submit(model, stream, input_buffer, output_buffer);

    //8. 获取结果并处理
    process_output(output_buffer);

    //9. 资源释放
    ES_SYS_MemFree(input_buffer);
    ES_SYS_MemFree(output_buffer);
    ES_NPU_DestroyStream(stream);
    ES_NPU_DestroyContext(ctx);
    ES_NPU_UnloadModel(model);
    ES_SYS_Deinit();
}
```

```
    return 0;
}
```

4.4. Python API 开发指南

Python API 封装了底层的 C 接口，提供了更加简洁、符合 Pythonic 习惯的用法，适用于算法验证和快速原型开发。

4.4.1. 核心开发流程 (Python)

Python API 的设计更加面向对象，核心流程如下：

1. **导入模块**： `import es_py`。
2. **系统初始化**：调用 `es_py.sys.init()`。
3. **设备设置**：调用 `npu.set_device(device_id)`。
4. **模型加载**：调用 `npu.load_model_from_file(path)`，返回一个 Model 对象。
5. **创建 Context/Stream**：通过 `npu.create_context()` 和 `npu.create_stream()` 创建对象。
6. **内存分配**：通过 `memory.memory_alloc()` 分配 Buffer。
7. **任务提交**：调用 `npu.submit([task])` 或 `npu.submit_async([task], stream)`。
8. **资源释放**：调用 `memory.memory_free()`，`npu.destroy_stream()`，`npu.destroy_context()`，`npu.unload_model()` 和 `es_py.sys.deinit()`。

4.4.2. 代码示例 (Python)

```
import es_py
import numpy as np

# 1. 系统初始化
es_py.sys.init()

# 2. 设置设备
es_py.npu.set_device(0)

# 3. 加载模型
model = es_py.npu.load_model_from_file("yolov3.model")

# 4. 创建 Context 和 Stream
ctx = es_py.npu.create_context()
stream = es_py.npu.create_stream(ctx)

# 5. 准备输入数据 (假设模型输入为 NHWC, dtype=float32)
input_data = np.random.rand(1, 416, 416, 3).astype(np.float32)

# 6. 分配内存并拷贝数据
input_buffer = es_py.memory.memory_alloc(input_data.nbytes)
input_buffer.copy_from_numpy(input_data)

# 7. 准备输出 Buffer
output_shape = (1, 255, 13, 13) # 示例形状
output_buffer = es_py.memory.memory_alloc(np.prod(output_shape) * 4)

# 8. 提交推理任务 (同步)
task = es_py.npu.Task(model, [input_buffer], [output_buffer])
es_py.npu.submit([task], stream)
```

```
# 9. 获取结果
output_np = output_buffer.to_numpy(dtype=np.float32).reshape(output_shape)
print("Inference result shape:", output_np.shape)

# 10. 资源释放
es_py.memory.memory_free(input_buffer)
es_py.memory.memory_free(output_buffer)
es_py.npu.destroy_stream(stream)
es_py.npu.destroy_context(ctx)
es_py.npu.unload_model(model)
es_py.sys.deinit()
```

4.5. 调试与性能评测

4.5.1. 日志调试

- **C 层**：默认输出 Error 级别日志，错误码可用于定位问题。
- **Python 层**：错误会转换为 `RuntimeError` 异常抛出，可通过 `npu.set_log_level()` 设置日志级别（如 DEBUG, INFO, ERROR）。

4.5.2. es_run_model 工具

为了方便用户快速评测模型，NPU Runtime 提供了 `es_run_model` 命令行工具，无需编写代码即可在开发板上测试模型速度和精度。

常用命令示例：

```
# 1. 单纯测试模型运行速度 (FPS)
/opt/eswin/bin/es_run_model -m /opt/eswin/data/npu/yolov3/ -r 1000

# 2. 指定输入/输出文件夹进行推理验证
/opt/eswin/bin/es_run_model -m /opt/eswin/data/npu/yolov3/ \
-i /opt/eswin/data/npu/yolov3/input/preprocessed/ \
-o /opt/eswin/data/npu/yolov3/output

# 3. 动态 Batch 测试
/opt/eswin/bin/es_run_model -m /opt/eswin/data/npu/yolov3/ -b 4 -r 1000
```


5. AcceleratorKit 开发指南

AcceleratorKit 是用于调用未编译入 ONNX 模型的算子的 API 库，如一些较为复杂的前处理、后处理算子等。AcceleratorKit 提供了可在不同类型设备上执行的计算加速功能，以便最大化发挥不同类型设备的计算加速能力。

5.1. 软件框架

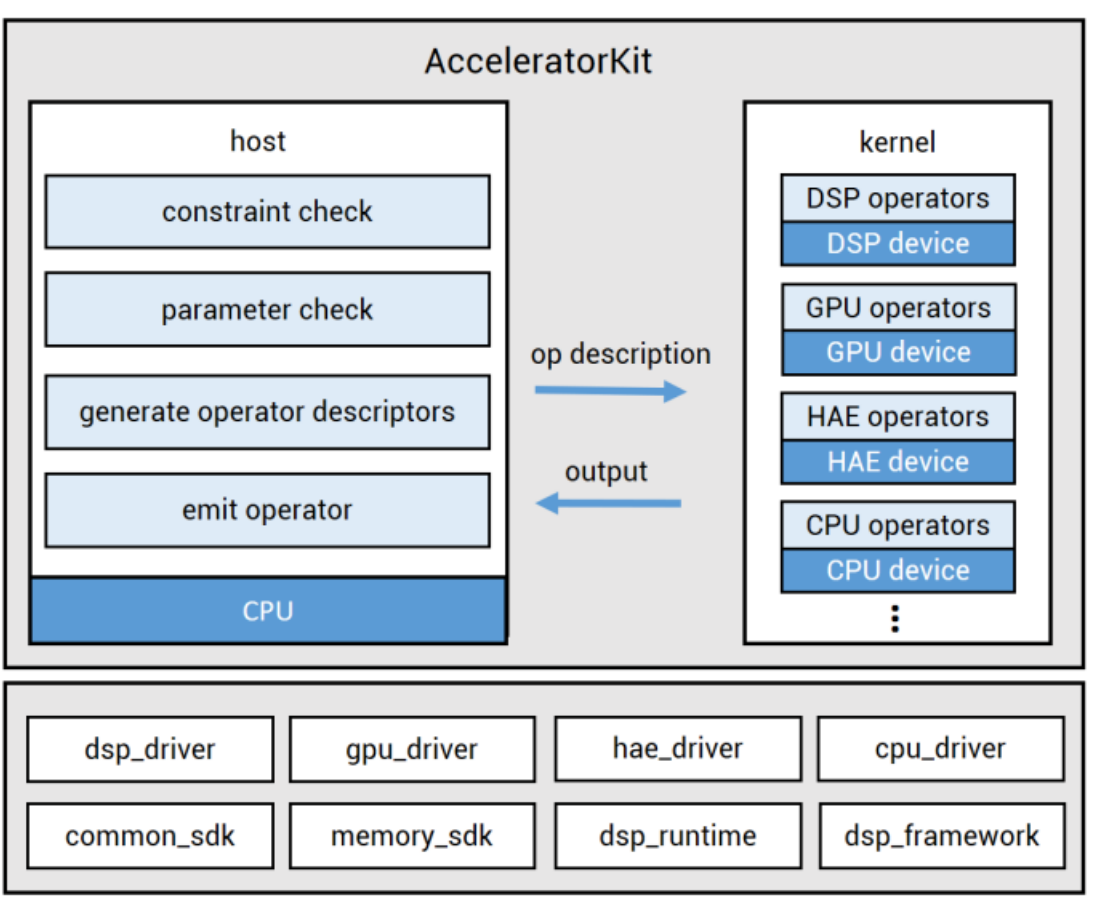


图 3: AcceleratorKit 软件架构

AcceleratorKit 当前支持的算子如下表所示：

算子名称	算子功能
ES_AK_DSP_CosDistance	计算两组数据之间的余弦距离
ES_AK_DSP_Argmax	对输入数据在指定维度进行最大排序
ES_AK_DSP_DetectionOut	对检测网络的输出 feature map 做后处理
ES_AK_DSP_Softmax	进行 Softmax 计算
ES_AK_DSP_PerspectiveAffine	对图像做透视变换
ES_AK_DSP_WarpAffine	对图像做仿射变换
ES_AK_CPU_SimilarityTransform	计算相似变换矩阵
ES_AK_DSP_RGB2HSV	将 RGB 图像转换为 HSV 图像
ES_AK_DSP_Sobel	计算图像在水平和垂直方向上的梯度

算子名称	算子功能
ES_AK_DSP_HoughLines	使用霍夫变换算法检测图像中的直线

5.2. 调用流程

5.2.1. 依赖模块

- **DSP 相关：** dsp_compiler , dsp_runtime , dsp_driver , dsp_fw , dsp_kernels 。
- **Common SDK：** 提供公共数据类型定义。
- **Memory SDK：** 提供跨设备内存管理工具。

5.2.2. 开发步骤

1. **获取版本与初始化：** 调用 ES_AK_GetVersion() 和 ES_AK_Init() 初始化硬件资源。
2. **设置设备：** 调用 ES_AK_SetDevice() 配置运算设备（如 DSP）。
3. **内存配置：** 调用 ES_SYS_MemAlloc() 为算子的输入/输出分配内存。
4. **执行算子：** 根据需求调用具体的算子接口，例如： ES_AK_DSP_DetectionOut 。
5. **资源释放：** 计算完成后调用 ES_SYS_MemFree() 释放内存，所有计算结束后调用 ES_AK_Deinit() 释放设备资源。

详细 API 定义及参数说明请参考 [ENNP 开发者手册](#)。

6. DSP 算子开发指南

本章介绍用户如何基于 DSP 硬件，自定义模型所需的特殊算子的完整操作流程。完整的自定义 DSP 算子开发流程如下：

- 搭建 ESWIN 交叉编译环境；
- 搭建 Cadence Vision Q7 DSP 编译环境；
- 开发自定义 DSP 算子；
- 编译自定义开发的 DSP 算子；
- 板端运行 DSP 算子。

注意：更为详细的开发指南，请参考 dsp_sample/kernel_sample 中的 README.md 和 sample 示例。

6.1. Sample 文件说明

ESWIN 提供了 dsp_sample/kernel_sample 示例，用于帮助用户快速上手自定义算子开发。具体目录结构如下所示：

```
kernel_sample
├── kernels
│   ├── bev_pool
│   │   ├── bev_pool.c
│   │   ├── bev_pool.h
│   │   └── CMakeLists.txt
│   ├── build.sh
│   ├── CMakeLists.txt
│   ├── include
│   │   ├── common
│   │   └── xtensa
│   └── user_add
│       ├── CMakeLists.txt
│       ├── user_add.c
│       └── user_add.h
├── README.md
├── test
│   ├── build.sh
│   ├── CMakeLists.txt
│   ├── data
│   │   ├── input_0.bin
│   │   └── input_1.bin
│   └── src
│       ├── common.c
│       ├── common.h
│       └── dsp_kernel_test.c
```

主要包括以下内容：

README.md：详细的开发和编译指南；

kernels：DSP 侧代码，以 user_add 算子为例，提供了 DSP 侧算子开发的 sample 示例；

test/src：host 侧代码，以调用 user_add 算子为例，提供了自定义算子的调用示例；

tesr/data：以二进制形式，存放了 user_add 所需的输入数据，用户可以使用该数据完成自定义算子 sample 自测。

6.2. 搭建 ESWIN 交叉编译环境

用户编译环境分 20.04 和 22.04 两种，区别在于生成交叉编译目录 rootfs，20.04 只能用 eswin 提供的 docker 编译生成 rootfs，22.04 可以直接在 x86 编译生成 rootfs。

以下都操作都基于 ubuntu 22.04。

6.2.1. 获取 riscv 交叉编译工具

```
wget https://github.com/riscv-collab/riscv-gnu-toolchain/releases/download/
↪ d/2024.04.12/riscv64-glibc-ubuntu-22.04-gcc-nightly-2024.04.12-nightly.tar.gz
sudo tar -xvf riscv64-glibc-ubuntu-22.04-gcc-nightly-2024.04.12-nightly.tar.gz -C /opt
```

注意：上述的 opt 目录，需要在后续的 host 编译环节和脚本保持一致。

6.2.2. 准备 eswin 交叉编译环境

- 参考 [BSP 快速入门指南](#) 中系统编译相关章节，并执行到“make_all”命令完成。
- 在 Host 执行

```
cd eswin-sdk-20241024/
sudo chmod 777 -R EIDS200B/
cd EIDS200B/output
mkdir rootfs
sudo apt-get install e2fsprogs
e2fsck -f root-EIDS200B-20241025-020530.ext4
resize2fs root-EIDS200B-20241025-020530.ext4 12G
sudo mount root-EIDS200B-20241025-020530.ext4 rootfs/

sudo chroot rootfs/
```

注意：20241025-020530 这类字符串是具体的发布版本信息，每次发布的版本号可能不同，请以实际获取的镜像文件版本为准。

- 在 rootfs 执行

```
root@ibudev12:/# cd home/eswin/
root@ibudev12:/# apt install -y gcc build-essential cmake
root@ibudev12:/# apt install es-sdk-dsp
root@ibudev12:/# apt install es-sdk-sample-dsp
root@ibudev12:/# apt install es-sdk-common
root@ibudev12:/# cd /usr/include/
root@ibudev12:/usr/include# ln -s riscv64-linux-gnu/bits bits
root@ibudev12:/usr/include# ln -s riscv64-linux-gnu/sys sys
root@ibudev12:/usr/include# ln -s riscv64-linux-gnu/gnu gnu
root@ibudev12:/usr/include# ln -s riscv64-linux-gnu/asm asm
root@ibudev12:/usr/include# cd /usr/lib
root@ibudev12:/usr/lib# cp riscv64-linux-gnu/* ./ -rf
root@ibudev12:/usr/lib# exit
```

6.3. Cadence Vision Q7 DSP 编译环境搭建

EIC7700 集成了 4 个 DSP 核，使用的是 Cadence 公司的 Vision Q7 DSP IP。Cadence DSP 开发环境安装可参考 Cadence 提供的官方文档《dev_tools_install_guide.pdf》。

注意：DSP 开发依赖 Cadence 提供的集成开发环境，需要正确设置 license 之后才可以使用 Xtensa Develop Tools，EIC7700 不提供 Cadence license，需要用户单独购买。

6.4. 自定义算子开发

6.4.1. host 侧准备

用户需要在 host 完成 op desc 生成、inputs 和 outputs 的内存申请和数据加载，其余步骤一般无需修改。

- op desc 填充方法参考 test/src/dsp_kernel_test.c 的 prepare_add_op_desc() 函数实现；

- inputs 的内存申请和数据加载参考 test/src/dsp_kernel_test.c 的 loadInputsData() 函数实现；
- outputs 的内存申请参考 test/src/dsp_kernel_test.c 的 prepareOutputBuffer() 函数实现。

6.4.2. DSP 侧算子开发

本手册简述 DSP 侧算子开发的常用接口，更为详细的开发指南和说明，请参考 dsp_sample/kernel_sample 中的 README.md 和提供的 sample 示例。

· DSP 侧算子函数接口

在 DSP 侧，共有 prepare、eval、get_prepare、get_eval 这 4 个函数，其函数名、参数均为固定接口，不能被修改。同时 prepare、get_prepare、get_eval 这 3 个函数的内容也不能被修改。用户只需要在 eval 函数中完成自己算子的实现代码即可。

· 获取片上内存起始地址及其大小

```
char *dram0_base_ptr = (char *)((ES_U32)get_available_dram0_base());
ES_U32 dram0_size = get_available_dram0_size();
```

提示

1. 在 DSP 开发时，不建议直接对 ddr 进行访存操作，可能会因为 cache 导致数据不一致，另外大量访存 ddr 也会极大拖慢算子的计算性能。建议使用 dma 将数据拷贝到 dram 上进行操作；
2. 每个 DSP 均有两个片上内存：dram0 和 dram1，两个 dram 的原始大小均为 128k。由于固件占用了部分空间，目前 dram0 起始地址为 0x28105000，可用空间为 108k；dram1 起始地址为 0x28120000，可用空间为 120k。

· 发起 dma 拷贝

```
idma_copy_3d_desc64(CHANNEL, &dst_ptr, &src_ptr, IDMA_DISABLE_INT, row_sz, nrows, ntiles,
↪ src_row_pitch, dst_row_pitch, src_tile_pitch, dst_tile_pitch);
```

· 等待 dma 拷贝结束

```
while (idma_hw_num_outstanding(CHANNEL) > 0) { }
```

· 日志打印

在 DSP 侧，请使用 es_printf 函数进行日志打印。

```
es_printf("this is eval of add op\n");
```

提示

1. 请登录串口，查看 DSP 侧的打印日志，例如 (下面命令仅供参考，具体命令请根据自身开发板连接方式进行调整)：

```
ssh 192.168.1.100 -l hostname
输入密码：password
sudo hostname -D /dev/ttyUSB3
```

登录串口后，再执行可执行程序调用算子，即可在串口获取到 es_printf 打印的内容。

2. 在 DSP 侧使用 es_printf 打印日志，会对算子性能造成较大影响。根据打印的日志长度，大致会导致耗时增加数十~数百微妙。

· DSP 性能优化

1. 将数据从 ddr 使用 dma 拷贝到 dram 进行计算，避免大量访问 ddr；
2. 建议优先使用 int8 数据类型，其次是 int16 和 f16 数据类型，尽可能不要使用 float32 或者 double 数据类型；
3. 使用 DSP 提供的 SIMD 指令，优化计算速度；
4. 使用时间戳，统计每部分耗时，对性能瓶颈部分进行针对性优化。

6.5. 算子编译

参考 dsp_sample/kernel_sample 中的 README.md，在算子编译前，根据实际情况更改“dsp_sample/kernel_sample/kernels/build.sh”函数“xtensa_env_setup”中的“xtensa 配置”。

注意：由于 XTENSA_CORE 和 XTENSA_TOOLS_VERSION 可能存在差异，如编译遇到问题请及时沟通解决。

- 编译 host 侧的测试程序

```
cd /path/to/your/dsp_sample/kernel_sample/test
./build.sh /path/to/your/eswin-sdk-20241024/EIDS200B/output/rootfs /path/to/your/kernel_sample/
↪ kernels/user_add
```

- 编译 DSP 侧的自定义算子

```
cd /path/to/your/dsp_sample/kernel_sample/kernels/
./build.sh /path/to/your/eswin-sdk-20241024/EIDS200B/output/rootfs
```

6.6. 自定义算子测试

- 将编译好的算子 es_dsp_kernel_add.pkg.lib 拷贝到开发板/lib/firmware/dsp_kernels/；
- 将测试程序 dsp_kernel_test 和测试文件 input_0.bin, input_1.bin 拷贝到开发板任意位置，如：/opt/eswin/；
- 执行测试命令：

```
cd /opt/eswin/
./dsp_kernel_test -c 0 -i input_0.bin input_1.bin
```

- 正确性验证：

执行结束后，会在 input_0.bin 所在目录，生成 output.bin。将其拷出到 Linux 下，使用：

```
md5sum output.bin
```

查看其 md5 码，如果其值为：4aae41d770845ae89f0cf33be15d0a77，则表示结果正确。

6.7. DSP 支持算子列表

算子名称	算子功能
Add	支持对两个输入进行加法操作
ArgMax	对输入求取最大值及其索引
AveragePool	对 kernel 范围内的数据求均值
BatchNormalization	对输入进行 BatchNormalization 操作
CropAndResize	对输入图片中提取特定区域进行缩放
Concat	对多个输入在指定轴上进行拼接
Conv	对输入进行卷积计算

算子名称	算子功能
Cos	计算输入的 Cos 数值
ConvTranspose	对输入进行反卷积计算
DepthwiseConv	对输入进行深度卷积计算
Focus	支持 yolo 检测网络中的 Focus 结构
Gather	从输入数据中收集 indices 对应的数据
Gelu	计算输入的 Gelu 激活函数输出
Gemm	对输入数据做矩阵乘
GlobalAveragePool	对输入数据每个通道分别求均值
HardSigmoid	计算输入的 HardSigmoid 激活函数输出
HardSwish	计算输入的 HardSwish 激活函数输出
InstanceNormalization	对输入进行 InstanceNormalization 操作
LayerNormalization	对输入进行 LayerNormalization 操作
Leaky 上 Relu	计算输入的 LeakyRelu 激活函数输出
Log	计算输入的 Log 数值
MatMul	对两个数据矩阵做矩阵乘
MaxPool	对 kernel 范围内的数据求最大值
Mish	计算输入的 Mish 激活函数输出
NonMaxSuppression	对检测网络的 boxes 进行 NMS 操作
Pad	对输入在不同 dims 上进行 Pad 操作
PRelu	计算输入的 PRelu 激活函数输出
ReduceMax	对输入在指定维度上求取最大值
ReduceSum	对输入在指定维度上求取累加和
Relu	计算输入的 Relu 激活函数输出
Relu6	计算输入的 Relu6 激活函数输出
Resize	对输入的张量数据进行缩放
RoiAlign	对输入的兴趣区域进行双线性插值
MaxRoiPool	执行 MaxRoiPool 池化操作
ROPE	在 LLM 模型 attention 模块前插入位置信息
Sigmoid	计算输入的 Sigmoid 激活函数输出
Silu	计算输入的 Silu 激活函数输出
Sin	计算输入的 Sin 数值
Slice	将单个输入根据 attribute 信息拆分为多个输出
Softmax	计算输入的 Softmax 作为输出
TopK	对输入在指定维度上求取前 K 个较大值及其索引
Transpose	根据 perm 参数，对输入做转置操作